North American Private University
IIT : International Institute of Technology
Département de Génie Informatique

AU : 2026-2027
Section : 1-GI

Exercice pratique — Déboguer un programme C
Algorithmique et Programmation en C

Vous allez écrire du code, le compiler, lire les messages du compilateur, et corriger les erreurs jusqu'à ce que le programme fonctionne et produise un résultat correct. C'est exactement le cycle que vous répéterez constamment en tant que programmeur.

Le programme suivant est censé lire trois notes d'examen, calculer leur moyenne, et afficher le total ainsi que la moyenne. Tel qu'il est écrit, il contient **8 erreurs** :

- Certaines sont des **erreurs de syntaxe** — le compilateur refusera de construire le programme tant qu'elles ne sont pas corrigées.
- D'autres sont des **erreurs silencieuses de conversion de type** — le programme compile et s'exécute, mais produit un résultat faux.

Trouver la seconde catégorie est plus difficile, car le compilateur ne les signale pas par défaut — il faut réfléchir aux types.

Étape 1 — Copier le code

Créez un nouveau fichier nommé `moyenne.c` et tapez le programme *exactement* comme montré ci-dessous, erreurs comprises. Ne corrigez rien pour l'instant.

```
#include <stdio.h>

int main(void)
{
    # Ce programme calcule la moyenne d'un etudiant
    int note1, note2, note3
    float 2moyenne;

    printf(Entrez trois notes d'examen : );
    scanf("%d %d %d", &note1, note2, &note3);

    moyenne = (note1 + note2 + note3) / 3;

    printf("Total = %d\n", note1 + note2 + note3;
    printf("Moyenne = %d\n", moyenne);

    return 0;
}
```

Étape 2 — Le compiler

Depuis un terminal, dans le dossier contenant `moyenne.c`, exécutez :

```
gcc -Wall moyenne.c -o moyenne
```

Que signifie cette commande

— `gcc` — invoque le compilateur GNU C Compiler.

— `-Wall` — active l'ensemble complet des *avertissements* courants ("Warnings : all"). Sans elle, gcc reste silencieux sur de nombreuses erreurs bien réelles mais techniquement légales en C, comme l'utilisation d'un mauvais spécificateur de format `printf`/`scanf` pour le type d'une variable, ou la lecture d'une variable non initialisée. `-Wall` ne crée aucune nouvelle erreur et n'empêche jamais la compilation à elle seule — elle fait simplement en sorte que gcc signale du code suspect dont il ne dirait sinon rien. **Prenez l'habitude de toujours compiler avec `-Wall`.**

— `moyenne.c` — le fichier source à compiler.

— `-o moyenne` — nomme l'exécutable produit `moyenne`. Sans `-o`, gcc le nommerait `a.out` par défaut.

Étape 3 — Corriger les erreurs, une par une

La première compilation échouera avec plusieurs messages d'erreur. Corrigez uniquement la *première* erreur affichée, puis relancez exactement la même commande de compilation. Répétez l'opération.

— Certains messages d'erreur pointent directement vers l'erreur.

— D'autres sembleront confus ou sembleront désigner une variable pourtant correcte — cela signifie généralement qu'une erreur *antérieure* (souvent juste au-dessus) a perturbé la compréhension du compilateur pour la suite. En cas de doute, regardez d'abord vers le haut.

Étape 4 — L'exécuter, et vérifier le résultat

Une fois que le programme compile sans erreur *et* sans avertissement, exécutez-le :

```
./moyenne
```

Testez-le avec plusieurs jeux de notes (essayez 12, 15, 10). Puis demandez-vous :

— La moyenne affichée est-elle réellement correcte ?

— Si non, quelle valeur a silencieusement changé de type quelque part, en perdant de l'information ?

Un programme qui compile sans aucun avertissement et qui donne malgré tout une réponse fausse a une **erreur de logique**, pas une erreur de syntaxe — la distinction la plus importante de tout cet exercice.